CONESTA FORGE — OPENING SPEECH

"Build Your Own AI Web Application"

Speaker: Ermars Castar, Conesta Event Coordinator

Estimated Duration: 45 Minutes

DELIVERY NOTES

- [PAUSE] = deliberate pause for effect or audience engagement

- (approx. X min) = section timing guidance

- Speak at a steady, clear pace — about 130 words per minute

- Make eye contact, move around the room naturally

- Day 1 content is detailed — take your time, do not rush it

PART 1 — WELCOME & INTRODUCTION

(approx. 4–5 minutes)

Good morning, everyone — and welcome.

My name is Ermars Castar, and I am the Conesta Event Coordinator. For the next five days, I will be right here alongside you — helping you, supporting you, answering your questions, and making sure that every single one of you leaves this event with something real, something you built with your own hands, something that actually works.

[PAUSE]

Let me just take a moment to look around the room. You have shown up. That already puts you ahead of a large number of people who had the same intention but never made it through the door. So before we say anything else — thank you for being here. That matters.

Now, I want to be upfront with you about something right from the start, because I think honesty at the beginning of a week like this is more valuable than any pep talk I could give you.

This week is going to push you. There will be moments where something does not work and you will not immediately know why. There will be moments where a concept feels slightly out of reach. There will be moments — probably around Day 3 or Day 4 — where the gap between what you imagined building and what currently exists on your screen feels uncomfortably wide.

That is completely normal. That is actually what learning looks like when it is real. And I want you to know that in those moments, you are not alone. That is exactly what I am here for. Come find me. Flag me down. Ask.

[PAUSE]

But here is the thing — and I genuinely believe this — I have watched beginners go through experiences exactly like this one, and I have seen what comes out the other end. By Friday afternoon, you will have a web application with a real, working AI feature, deployed at a public URL that anyone in the world can open on their phone. That is not a small thing. That is something most people never do.

So let us get into it.

PART 2 — ABOUT THIS COURSE

(approx. 4–5 minutes)

The name of this course is "Build Your Own AI Web Application," and I want to be very clear about what that means, because those words are doing a lot of work.

"Build Your Own" — this is not a watch-someone-else-do-it course. You are not here to observe. You are here to produce. By the end of Day 1, you will have written a spec and scaffolded a running app. By the end of Day 2, your app will be on the public internet. By the end of this week, it will do something genuinely intelligent with your own data.

"AI Web Application" — not a toy, not a tutorial exercise, not something that only works on your laptop. A real web application, connected to a real cloud database, calling a real AI model, deployed to a real URL.

[PAUSE]

Here is how the week is structured. Five days, two lessons per day — ten lessons in total. Each day has a theme, and each day ends with a concrete deliverable that you have actually shipped. Not planned, not designed — shipped.

Day 1 is Spec and Foundation.
Day 2 is Build the Core App.

Day 3 is Add the Brain.
Day 4 is Build Your AI Feature.
Day 5 is Polish and Ship.

And here is something important about how the lessons are structured. Every lesson ends with a required video — a short two-to-three minute recap. That video is a gate. You cannot complete the lesson until you have watched it through. This is intentional. It is not bureaucracy. It is a mechanism that protects you from moving on before you are ready.

The course is also designed for beginners. No prior web development experience. No prior AI experience. The assumption is only that you can follow instructions and are comfortable installing software. If that is you, then you are exactly who this course was made for.

PART 3 — DAY 1 IN DEPTH: SPEC AND FOUNDATION

(approx. 25 minutes)

[PAUSE — take a breath, shift tone slightly more deliberate]

Now I want to spend significant time on Day 1, because Day 1 is the most important day of the week. Not because the technical content is the hardest — it is actually the most straightforward of all five days. Day 1 is the most important because the decisions you make today determine whether the rest of the week goes smoothly or whether you spend every subsequent day fighting against a foundation that was never properly laid.

Get Day 1 right, and Days 2 through 5 feel like building. Get Day 1 wrong, and Days 2 through 5 feel like fixing. So let us get it right.

Day 1 has two lessons. Lesson 1.1 is Setup and Your One-Page Spec. Lesson 1.2 is Scaffolding a Web App with AI. Let us go through both in detail.

THE STACK

(approx. 8 minutes)

Before we talk about what you are building, let us talk about what you are building it with. Because this course does something that I think is genuinely helpful for beginners, and I want you to understand why.

The course gives you one stack. One fixed, known-good set of tools. You do not have to choose. You do not have to research. You do not have to wonder whether you picked the right framework or the right database. The decision has already been made for you, and the reason it was made for you is not because there is only one right answer — there are many good stacks in the world — but because spending your first day evaluating technology options is a day you could have spent building something.

So here is the stack. There are four layers.

[PAUSE]

Layer one is the Framework. The tool is Next.js.

Next.js is built on top of React, which is the most widely used UI library in the world. But Next.js does something that React alone does not do — it combines your front end and your back end in a single project. What that means is this: the pages that your users see and interact with, and the server-side code that handles data and API calls — all of it lives in one project, one folder on your machine, one GitHub repository. There is no separate backend server to set up. There is no separate deployment to manage. One project.

Next.js uses something called the App Router, which is a file-based routing system. This means that the structure of your folders directly maps to the structure of your web pages and your API endpoints. If you create a folder called "habits" inside the "app" folder, you get a page at the URL slash habits. If you create a folder inside "api" called "habits" with a file called "route.ts" inside it, you get an API endpoint at slash api slash habits. The file structure is the map of your application.

Next.js is also written in TypeScript, which is a version of JavaScript that adds type safety — meaning it helps catch certain kinds of mistakes before your code ever runs. You do not need to know TypeScript deeply to use it in this course. The AI coding assistant you will be working with knows it extremely well.

[PAUSE]

Layer two is the Database. The tool is MongoDB Atlas, used with a library called Mongoose.

MongoDB is a document database. Unlike a traditional spreadsheet-style database that organizes data into rigid rows and columns, MongoDB stores data as flexible JSON-like documents — objects that look very similar to the JavaScript objects you will already be writing in your app. This makes it a natural fit for JavaScript and TypeScript projects.

MongoDB Atlas is the cloud-hosted version of MongoDB. You do not install anything on your machine. You create a free account, create a free cluster — which is essentially your database server — and MongoDB runs that server for you in the cloud. The free tier, called M0, is more than enough for a five-day build.

Mongoose is the library that sits between your Next.js code and your MongoDB database. You use Mongoose to define a schema — which is the named list of fields for your data entity — and from that schema you get a model. The model gives you clean methods to create documents, read documents, update documents, and delete documents. We will come back to those four operations in more detail when we talk about Day 2.

The connection between your app and your MongoDB Atlas database is handled by a secret string called a connection URI. It looks something like `"mongodb+srv://username:password@cluster.mongodb.net/yourapp". That string` contains your database username and password, which is why it is a secret. You never write it into your code. You never commit it to GitHub. We will talk about where it actually lives in a moment.

[PAUSE]

Layer three is Deployment. The tool is Vercel.

Vercel is the company that created Next.js, and it is also the easiest place to deploy a Next.js application. The way it works is beautifully simple. You connect your GitHub repository to Vercel, and from that moment on, every time you push code to GitHub, Vercel automatically deploys it. Your app gets a public URL immediately — something like yourapp.vercel.app. Anyone in the world can open that URL and use your application.

What makes Vercel particularly valuable is that it handles a lot of the infrastructure complexity that would otherwise take days or weeks to configure. You do not worry about servers, you do not worry about scaling, you do not

worry about certificates or HTTPS. Vercel handles all of that. You push code, your app is live.

There is one important thing to know about Vercel upfront, and we will return to this on Day 2. When you deploy to Vercel, you need to tell Vercel about your environment variables — including your database connection string and your AI API key. Vercel cannot read your local .env.local file. You have to enter those values directly into the Vercel dashboard under your project settings. This is one of the most common sources of deployment failures, and knowing it exists before it happens to you is valuable.

[PAUSE]

Layer four is the AI. The tool is Groq, used with the groq-sdk library.

Groq is an AI provider. What that means is: Groq runs large language models — the same kind of AI model that powers tools like ChatGPT — and exposes them through an API that your application can call. You send Groq a prompt, and Groq sends back a generated text response.

The specific model this course uses is called llama-3.3-70b-versatile. It is an open-source model — which means no single company owns it — that Groq hosts and makes available through their API. Groq's free tier is genuinely generous. For a five-day project, you will not run out of credits under normal usage.

The groq-sdk is a JavaScript library that handles the communication with Groq's API. You install it into your project with one command, and it gives you a clean client object that you use to make calls to the model.

Now — and this is critical — Groq calls must happen on the server side of your application. Your API key is a secret. If you make a Groq call from the browser — from client-side JavaScript — your API key is exposed to anyone who opens your app and looks at the network tab in their browser. That is a leaked key, and a leaked key means anyone can make calls billed to your account. All Groq calls go in your Next.js API routes. Server side, always.

[PAUSE]

In addition to those four main layers, there are three supporting tools.

Git is version control software that tracks every change you make to your code. Think of it as a save system with unlimited history — you can always go back to a previous state. Git is installed on your machine.

GitHub is the cloud-hosted platform where your Git repository lives. Your code gets pushed to GitHub, and that GitHub repository is the one Vercel reads to deploy your app. The judge at the end of the event also reads your GitHub repository. Everything you build lives there.

An AI coding assistant is the tool you will use to write most of the actual code. This course works with Cursor, GitHub Copilot, or Claude/ChatGPT. These tools can take plain-English descriptions of what you want and generate working code. One of the most important skills this course teaches is how to direct these tools effectively. We will come back to that in Lesson 1.2.

[PAUSE]

One final thing about the stack. This application has no authentication. No login, no users, no passwords. It is intentionally designed as a single-user build. The reason is simple: authentication adds significant complexity, and in five days, complexity is the enemy. The goal is a working, deployed AI feature by Friday. Authentication can come after the event.

LESSON 1.1 — SETUP AND YOUR ONE-PAGE SPEC

(approx. 9 minutes)

Let us talk about Lesson 1.1. Its hook is this: "Nobody fails this event because they can't code. They fail because they burned a day fighting an install, or never decided what they were building."

Read that again. Nobody fails because they cannot code. They fail because of installs and indecision. Both of those are things you can control today.

[PAUSE]

First, the practical setup. There are four free accounts you need to create, and two things you need to install on your machine.

The accounts are: GitHub, MongoDB Atlas, Vercel, and Groq. All four are free. All four are essential. You cannot complete this course without all four. If you do not have them, the most important thing you can do in the

next hour is create them.

The installations are: Git and Node.js. Git is the version control software I mentioned. Node.js is the runtime that runs your Next.js application on your local machine. When you install Node.js, you also get a tool called npm — Node Package Manager — which is what you use to install JavaScript libraries like groq-sdk and Mongoose. Always download the LTS version of Node.js — LTS stands for Long Term Support, and it is the most stable version.

You also need an empty GitHub repository. Create one now if you have not already. Give it a name that reflects your project. This repository is the home of everything you build this week.

[PAUSE]

Now. The spec. And I want you to hear me clearly on this, because it is the most important thing I will say before lunch.

The one-page spec is the highest-leverage fifteen minutes of the entire week.

Not writing code. Not setting up tools. Not configuring anything. Writing your spec. Because the spec is the frozen contract. It is the document that answers — once, clearly, with finality — what you are building. And once you have written it, you stop renegotiating it.

Here is why this matters so much. This event is five days long. Five days sounds like plenty of time until you start losing hours to scope creep — until the app you were building at 9am on Monday has grown by Tuesday afternoon to include a calendar feature, a chat interface, a leaderboard, and a social sharing system. I have seen it happen. The app that tries to do everything ships nothing.

Your spec is the wall that protects you from that.

[PAUSE]

What goes in a one-page spec? Five short sections.

Section one: the title and one-sentence pitch. Name your app. Describe it in one sentence. "X lets you [do something] so that [some benefit]." That is it. If you cannot say what your app does in one sentence, the app is not focused enough yet.

Section two: the user. Who is this for? In a single-user, no-auth build, the

answer is often simply "me" or "anyone who wants to track [thing]." You still need to be able to name this person, because it shapes every subsequent decision.

Section three: the data. What are your one or two core entities? An entity is the main "thing" your app stores. If you are building a habit tracker, the entity is a habit. If you are building a recipe manager, the entity is a recipe. Name your entity and list its fields. A habit might have: a name, a streak count, a target frequency, and a created-at timestamp. Four or five fields is plenty. More than that is over-scope.

Section four: the screens. What are the pages of your application? List them. A home page, a detail page, maybe a form page. You do not need to design them — just name them. Knowing what screens exist tells you what your app needs to do.

Section five: the AI feature. This is the sentence the course cares most about. "The model takes [blank] and returns [blank]." Fill in those blanks. "The model takes my last seven habit entries and returns a personalized reflection on my progress." "The model takes a recipe's ingredient list and returns suggested preparation notes." One feature. One job. Specific.

[PAUSE]

If your spec spills past one page, it is too big. Cut it. The discipline of fitting it on one page is not an arbitrary constraint — it is the exercise of scoping properly. A one-page spec is not a small ambition. It is a focused one.

Now, I want everyone in this room to do something. When you write that one-page spec, write this sentence in particular — by hand if you need to:

"The model takes _____ and returns _____."

Fill it in. Do not leave it vague. Do not write "the model does AI stuff." Name the input. Name the output. That sentence is the nucleus of your entire AI feature, and if it is fuzzy now, it will be three times harder to build correctly on Day 4.

[PAUSE]

The knowledge check questions for Lesson 1.1 are worth knowing in advance,

because they reinforce the most important concepts.

The course's default stack is Next.js, MongoDB Atlas with Mongoose, Vercel, and Groq. Swap any of those only if you already have strong experience with an alternative.

The four accounts are GitHub, MongoDB Atlas, Vercel, and Groq. Node and Git are installed tools, not accounts. Know the difference.

To "freeze the contract" means to make the core decisions once and stop renegotiating them mid-build. Not perfect — decided. There is a huge difference between those two things.

And the one-page spec contains your data entities and their fields. Not your styling choices. Not your model configuration. Not your API keys. Entities and fields.

LESSON 1.2 — SCAFFOLDING A WEB APP WITH AI

(approx. 8 minutes)

Lesson 1.2 is where you go from a spec on paper to a running application on your screen. Its hook is this: "An AI builder is a power tool. Point it precisely and it scaffolds in minutes; wave it vaguely and you get a generic mess you have to throw away."

Let us start with the scaffold itself.

You create a new Next.js application with a single command:

```
npx create-next-app
```

[PAUSE]

That is it. Run that command, answer a few prompts — yes to TypeScript, yes to the App Router, yes to Tailwind CSS if you want styling — and within a minute or two you have a complete, runnable project. The folder structure is generated for you. The configuration files are there. The development server is ready to start. Type "npm run dev" and open your browser at localhost:3000, and you have a running web application.

That is the power of modern tooling. You did not write a single line of code and you already have a foundation. Now the job is to fill it with the content your spec describes.

[PAUSE]

This is where your AI coding assistant comes in, and this is where the skill of prompting becomes critical. I want to spend a few minutes on this because it is genuinely the skill that separates learners who feel like AI makes them faster from learners who feel like AI makes a mess and they spend more time cleaning it up than they saved.

A weak prompt to your AI assistant sounds like this: "Build me a habit tracker." That prompt is going to get you a generic, one-size-fits-all result. The AI has to guess everything — what fields your habit has, what screens you want, what your data looks like, how your API is structured. It will guess, and it will guess something reasonable, and it will not be what you actually need. You will spend more time deleting and rewriting than you would have spent writing it yourself.

A strong prompt sounds like this: "Create a page at `app/habits/page.tsx that fetches from GET /api/habits and renders each` habit as a card showing the name and the current streak. Use TypeScript. The habit has fields: name (string), streak (number), and target (number)."

[PAUSE]

Notice what that prompt does. It names the exact file path. It names the API endpoint the page should call. It describes the component it wants — a card. It names the specific fields to display. It specifies the language. That is a prompt that gives your AI assistant everything it needs to produce something useful on the first try.

The formula for a strong prompt is four things: the file path, the data source, the data shape, and the boundary — meaning one screen, one component, one endpoint at a time. Do not ask the AI to "build everything." Ask it to build one thing, review it, confirm it works, and then ask for the next thing. This is additive building. Small pieces, each one tested, stacked on top of each other.

[PAUSE]

Now, here is what your Day 1 milestone should look like. And I want to be explicit about this because some of you will be tempted to spend the afternoon making your app look beautiful instead.

By end of Day 1, your app should boot. Your core screens should be navigable. At least one screen should show real data — even if that data is hard-coded for now. Your GitHub repository should have a committed version of this skeleton. And your README file should state your spec.

That is it. Not beautiful. Not finished. Not perfect. Boots, navigates, shows data, committed, README exists. That is a passing Day 1.

A grey, ugly skeleton that runs beats a beautiful screen that does not boot. Every time. Without exception. Polish is Day 5's job, and you cannot polish something that does not exist.

[PAUSE]

The knowledge checks for Lesson 1.2 reinforce this precisely.

"npx create-next-app" gives you a runnable project with both UI pages and backend API routes in one place. That is the answer, and it is the foundation of everything.

The stronger prompt is always the specific one — file path, data source, data shape, one screen at a time.

The right Day 1 milestone is: boots, navigable core screens, at least one real-data screen. Not finished, not styled — functional skeleton.

And the Day 1 rubric requires a scaffolded, runnable app skeleton plus a README stating the spec and screens. Not a finished app. A skeleton with a README.

[PAUSE]

Before we move on, I want to come back to one practical thing you must do today. Your README file. It does not need to be long. It does not need to be pretty. But it needs to exist, and it needs to state, in plain language: what your app is called, who it is for, what it does, and what your AI feature is. That is the minimum. The judge on Day 5 reads your README before they look at anything else. Give them something clear to read.

PART 4 — DAYS 2 THROUGH 5 — OVERVIEW

(approx. 10 minutes)

[PAUSE — shift tone to slightly faster-paced, overview mode]

Now I am going to walk you through the remaining four days at a higher level. You will go deep on each of these as the week unfolds. My goal right now is to give you the map — to help you see where you are going so that when you are in the middle of Day 3 wondering how the pieces connect, you already have the picture.

DAY 2 — BUILD THE CORE APP

Day 2's theme is: make the app store real data and put it on the public internet. That is the whole job of Day 2, and it is a significant one.

Lesson 2.1 is Data and CRUD with MongoDB. You will take the entity you defined in your spec yesterday and build a proper Mongoose schema for it. You will connect your Next.js app to your MongoDB Atlas cluster using your MONGODB_URI secret. And you will implement the four fundamental operations that almost every web application is built on: Create, Read, Update, and Delete — CRUD.

In Next.js, these operations live in your API route files. A file at

```
app/api/habits/route.ts handles GET requests — which read your habits list —
```

and POST requests — which create a new habit. A file at

```
app/api/habits/[id]/route.ts handles PATCH requests to update a specific
```

habit and DELETE requests to remove one.

Two specific things you need to watch out for on Day 2. First, the Mongoose model guard line. When you define a Mongoose model, you write it like this: "mongoose.models.Habit || mongoose.model('Habit', HabitSchema)". That double-check prevents errors during hot reload — when Next.js restarts your code automatically after changes. Skip that guard and you will get a confusing error about a model already being registered. Second, the cached database connection. Because Next.js API routes can run in a serverless environment — where each request might start a fresh process — you need to cache your MongoDB connection so you are not opening a new database connection on every single API call. The course provides the pattern for this.

Lesson 2.2 is Deploy to Vercel. And the lesson's hook says it better than

I can: "An app on your laptop is a science project. An app with a public URL is something a stranger can actually use — and a judge can actually open."

You will connect your GitHub repo to Vercel, configure your environment variables in the Vercel dashboard — and I am telling you now, do not forget this step, it is the number one cause of deployment failures — and get a live URL. Test that live URL. Create data through it. Confirm the data persists. If it breaks, debug it now while it is early and low-stakes. Do not save deployment debugging for Day 5.

DAY 3 — ADD THE BRAIN

Day 3's theme is: get an AI model talking to your application.

Up until Day 3, your app is a perfectly functional web application with no AI. Day 3 changes that. Lesson 3.1 is Getting an LLM Into Your App. You will install the groq-sdk library, create your API key in the Groq console, store it in your .env.local file as GROQ_API_KEY, and write your first Next.js API route that actually calls the Groq model and returns a response.

The Day 3 rubric checks for three things: groq-sdk is imported in package.json, the key is read from an environment variable, and there is a real call site that returns model text. All three need to be true.

Lesson 3.2 is Prompts and Key Safety. This lesson teaches you two things that compound in importance as the week goes on.

The first is prompt anatomy. A good prompt has three parts: a role, a specific task, and the input data. The role tells the model who it is — "You are a fitness coach helping the user reflect on their habits." The task tells it exactly what to do — "Write a short, encouraging reflection on the following habit data." The input is the actual data — whatever came out of your MongoDB database for this user.

The second is key safety. Your Groq API key must never appear in your code, in your README, or in any file that gets committed to GitHub. It lives in .env.local, which is gitignored. If you ever accidentally commit a key, the correct response is to immediately revoke it in the Groq console and generate a new one. Gitignoring a committed key does not un-leak it. The history

still contains it. Revoke and replace.

DAY 4 — BUILD YOUR AI FEATURE

Day 4 is the day your app earns its name.

Lesson 4.1 is Designing Your Signature Feature. The course describes four archetypes — four shapes of app — and each one maps naturally to a specific kind of AI feature. If you are building a Feed or Board, your AI feature probably transforms or enriches a post. If you are building a Tracker or Log, your AI feature probably reflects on your entries over time. If you are building a Catalog or Directory, your AI feature probably enriches or matches items. If you are building a Chat or Assistant, your AI feature is the conversation itself.

Whatever your archetype, the key principle is: your AI feature must run on real user data. Not hard-coded sample text. Real data that the user created inside your app, read live from MongoDB. If the input is hard-coded, it is not a feature — it is a demo that will never generalize.

Lesson 4.2 is Wiring the Feature End-to-End. This is about connecting every link in the chain. The request travels from the browser, to your Next.js API route, to Groq, back to the route, back to the browser, and onto the screen. That path of four links is also your debugging map. When something goes wrong, you trace the path and find where the break is.

Two practical things for Day 4. First, always show a loading state while your AI call is running. Model calls are not instant. Without a loading indicator, users will click the button again, assuming it did not work. A simple "thinking..." message is the difference between an app that feels alive and one that feels broken. Second, if your app is a Chat archetype — a conversational assistant — remember that language models are stateless. They do not remember previous messages. You have to send the entire conversation history with every request. Keep the last six to ten turns, not the entire history, or your requests will become enormous.

DAY 5 — POLISH AND SHIP

Day 5's theme is: make it reliable, then make it real.

Lesson 5.1 is Guardrails and Reliability. Anything that calls an AI model and the internet will sometimes fail. The difference between a demo and a product is what happens on a bad day. You will wrap your Groq calls in try-catch blocks to handle errors gracefully. You will validate the output you get back from the model before you try to render it. You will give every screen three states: an empty state for when there is no data yet, a loading state for when something is happening, and an error state for when something went wrong.

Lesson 5.2 is Ship and Share. This is the finish line. You will confirm your deployed app works end-to-end from a fresh browser window. You will complete your README — the final version should include your app's pitch, a prominent link to the live demo, step-by-step instructions for running the project locally with environment variable names listed (but never their actual values), and an explanation of how your AI feature works. You will record a thirty-to-sixty second demo clip showing the critical path: open the app, perform the core action, trigger the AI feature, show the result. And you will write a brief share post — what you built, what the AI feature does, what you learned, and a link to the live demo.

The Day 5 rubric is: error handling and guardrails on the AI call, a working deployed demo, a clear README, and a share post. All four.

PART 5 — CLOSING

(approx. 3 minutes)

[PAUSE — slow down, bring it home]

I want to leave you with a few thoughts before we get into the work.

This course is designed to make one thing obvious by Friday: shipping is a skill. It is not a talent some people are born with. It is a skill, and like any skill, it is developed by doing it. This week, you are going to do it. Once. And once you have done it once, the second time is easier. The third time is faster. Eventually it becomes natural.

The lesson I would ask you to carry with you through every single day this week is this: a running, deployed, imperfect thing is worth more than a

perfect thing that never ships. Perfection is a trap. Ship, then improve.

[PAUSE]

The stack you are learning this week — Next.js, MongoDB Atlas, Vercel, Groq — is not a beginner's toy. These are production tools used by real companies shipping real products to real users. The skills you build here transfer. The patterns you learn here apply. You are not learning a simplified version of real web development. You are doing real web development, with training wheels in the right places.

[PAUSE]

A few practical reminders as we begin Day 1.

Create your four accounts now if you have not already. GitHub, MongoDB Atlas, Vercel, Groq.

Install Git and Node.js LTS on your machine.

Write your spec today. All of it. Including the sentence: "The model takes _____ and returns _____." Write it down.

And if at any point you are stuck — truly stuck, not just momentarily confused — come find me. That is what I am here for. I am Ermars Castar, I am your Conesta Event Coordinator, and I am not going anywhere.

[PAUSE]

Let us build something.

Thank you.

[END]

APPROXIMATE TIMING SUMMARY

Part 1 — Welcome & Introduction ~4-5 min
Part 2 — About This Course ~4-5 min
Part 3 — Day 1 In Depth
The Stack (4 layers + supporting tools) ~8 min
Lesson 1.1 — Setup & One-Page Spec ~9 min
Lesson 1.2 — Scaffolding with AI ~8 min
Part 4 — Days 2-5 Overview ~10 min

Part 5 — Closing ~3 min

TOTAL ~45 min